



# TrueVision - Beta Release

Minsu Pak, Yoonseong Kim



## Delays...

- Exams and other writing assignments
- We wanted to put in some security in our project



# Main Use Cases

## Use case 1: Submit Image

**Actor:** User

**Priority:** Essential

**Primary Scenario:**

1. The user clicks on “select file” ✓
2. The user selects a file ✓
3. The user clicks on upload ✓
4. The file is sent to the backend ✓

**Extension Scenario:**

2a. The user selects a file that is not an image, and the website rejects it. ✓

## Use case 2: Evaluate Image

**Actor:** Backend

**Secondary Actor:** AI model

**Priority:** Essential

**Primary Scenario:**

1. The backend receives a file ✓
2. The backend checks if the file is an image ✗
3. The backend preprocesses the image ✓
4. The backend passes the image through an AI model ✓
5. The backend sends the result back to the user ✓

**Extension Scenario:**

2a. The backend sends the user an error message ✓

# Main Use Cases

## Use case 3: Select Model

**Actor:** User

**Priority:** Essential

**Primary Scenario:**

1. The user gets an option to pick a model ✓
2. The user selects a model ✓
3. The choice is saved on the website ✓

**Extension Scenario:**

2a. If the user is not logged in, the user cannot select a model ✗

## Use case 4: Purchase a Paid Subscription

**Actor:** User

**Secondary Actors:** Backend, Database

**Priority:** Expected

**Primary Scenario:**

1. The user is logged in ✓
2. The user clicks to purchase the paid plan
3. The user completes the steps to purchase the paid plan
4. The backend receives data that a user has purchased a paid plan
5. The backend authenticates the user's account is real and that their account information is valid through the data stored in the database
6. The database information is altered to show that the user is a paid user
7. The user is shown that their purchase was successful

**Extension Scenario:**

1a. If the user is not logged in, redirect to the login page

5a. If there are issues with the user's account data, an error is thrown, and the user is notified of the error. The payment and change of account status are cancelled.

# Main Use Cases

## Use case 5: Generate API Key

**Actor:** User

**Secondary Actor:** Backend, Database

**Priority:** Nice to have

### Primary Scenario:

1. The user is logged in and has a paid subscription
2. The user navigates to their account settings page ✓
3. The user is presented with information regarding the management of the API key ✓
4. The user clicks a button to get an API key
5. The backend receives data that the user wants an API key
6. The backend authenticates the user's account is real and that their account information is valid through the data stored in the database ✓
7. The backend checks that the user is a paid user through the database information
8. The backend checks if the account already has an associated API key
9. If an API key already exists, it sends the key information to the website
10. If an API key does not exist, a key is generated, and the key information is sent to the website
11. When the key information is received, the user is shown their API key

### Extension Scenario:

- 3a. If the user is not a paid user, then no API key information is shown, and instead, there is a button to prompt the user to purchase a subscription ✓
- 6a. If there are issues with the user's account data, an error is thrown, and the user is notified of the error. The payment and change of account status are cancelled.
- 7a. If the database information shows the user is not a paid user, but they were able to access the API key management features, this is a sign of a major error, and the user is shown that there is an issue with their account, and an API key is not available



# Our (Totally Smart) Security

Based off Kerberos



# Our (Totally Smart) Security

- Our backend acts as a **Trusted Third Party (TTP)**
- Additionally, it acts as a **Key Distribution Center (KDC)**

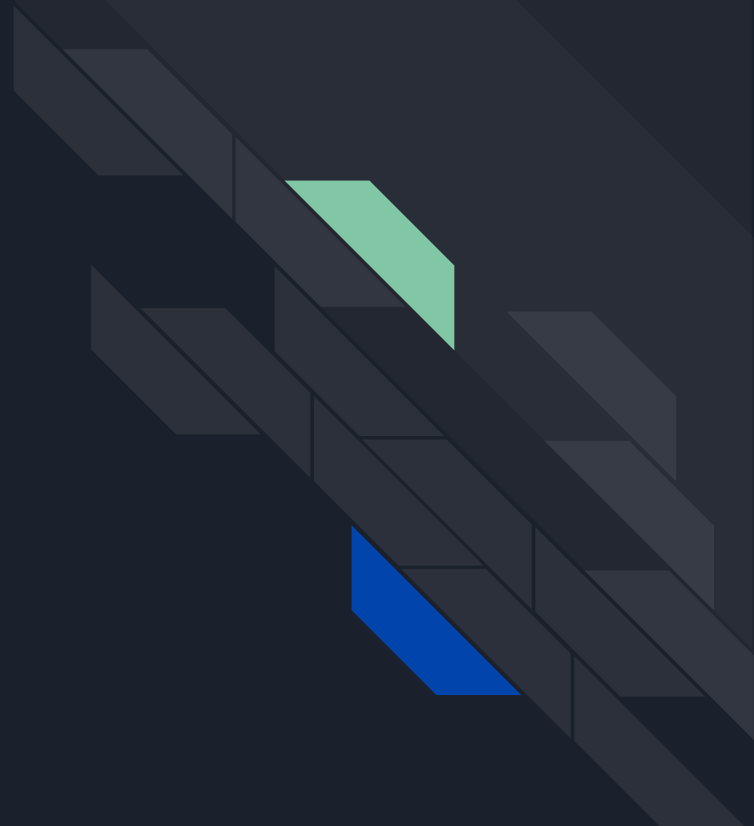


# Our (Totally Smart) Security

- The reason why we did this is to keep our backend as stateless as possible



Beta Demo





# If you wish to follow along

THIS ONE FIRST



<https://220.126.157.140:10000/docs>

This one is actual site



<https://yoonseongkim2000.github.io/CSE-416-Team3/>



# Development Challenges

- CORS..... (Cross-Origin Resource Sharing)
  - Frontend & Backend different hosts
  - Setting up Cookies for user auth & persistent login
  - As of **very** recently, browser (Chrome, Firefox) have started restricting third-party Cookies
  - Third Cookies need a “Partitioned” attribute to save on user end
  - NOT possible with our Python 3.11 & FastAPI versions
- **Solution:** Fixed by manually spoofing the attributes



# Development Challenges

- User Authentication & Persistent Login
  - Using access & resource tokens
  - FastAPI (backend) & HashRouter (React/frontend)
- **Solution:** Slowly implementing & testing solutions, reading LOTS of documentation...



# Future Work

## **Minsu:**

- User account settings functions (frontend, backend, database)
- Paid plan functionality (frontend, backend, database)
- Proper persistent login / authorization (frontend)

## **Andy:**

- Finish user API call function (backend)
- Token purchase and generation (frontend, backend)